

Lakeflow Jobs anatomy and task types

Jobs, tasks and the DAG; the task types the exam names; compute per task; and passing values between them.

A job is a **graph of tasks**, not a script. Each task has its own type, its own compute, and its own place in the dependency graph.

The anatomy

01

Job

The unit you schedule, own and monitor.

02

Tasks

One or more.
Each has a type and a compute resource.

03

Dependencies

Each task names the tasks it waits for.

04

The DAG

Databricks derives order and parallelism from those dependencies.

05

Runs

Each execution is a run, with per-task status and history.

DECLARE, DON'T SEQUENCE

You never write the order. You declare dependencies and the order follows. Anything undeclared may run in parallel.

Task types

TYPE	WHAT IT RUNS
Notebook	A notebook from the workspace or a Git folder
SQL	A query, a dashboard, an alert, or a SQL file
Pipeline	A Lakeflow declarative pipeline. Triggered only.
Python script / Python wheel	A .py file, or a packaged wheel
dbt	A dbt project against a SQL warehouse
JAR / Spark Submit	Compiled Scala or Java work
Run job	Another job, as a task inside this one
If/else and For each	Control flow rather than work. Module 17.

ON THE EXAM

The exam guide names four: notebook, SQL query, dashboard and pipeline. Know those precisely, and the others by sight.

The notebook task

- ◆ **The most common type.** Runs a notebook, with parameters passed in as widgets.
- ◆ **Source can be the workspace or a Git folder,** which is how notebooks get version-controlled deployment.
- ◆ **Recommended compute is serverless jobs;** classic jobs and classic all-purpose are also supported.
- ◆ **All-purpose compute is supported but discouraged,** for the reasons module 2 covered.

ON THE EXAM

Reading parameters with `dbutils.widgets.get` is how a notebook task receives values from the job.

The SQL task

- ◆ **Four things it can run:** a saved query, a dashboard, an alert, or a SQL file.
- ◆ **Runs on a SQL warehouse**, not on a cluster. Serverless is recommended; pro is supported.
- ◆ **Classic SQL warehouses are not supported** for SQL tasks.
- ◆ **A dashboard task refreshes the dashboard**, which is how a dashboard gets updated on a schedule after a pipeline.

ON THE EXAM

The dashboard task is why the exam guide names dashboards. It is a task type, not a Databricks SQL topic in its own right.

The pipeline task

- ◆ **The pipeline must be a triggered pipeline.** Continuous pipelines are not supported as a job task.
- ◆ **The reasoning:** a continuous pipeline is already running, so triggering it would be redundant.
- ◆ **You can optionally trigger a full refresh** from the task.
- ◆ **Parameter overrides can be set** on the task, without editing the pipeline.

ON THE EXAM

Jobs are procedural: they define relationships between tasks. Pipelines are declarative: they define relationships between datasets.

Two more worth knowing

- ◆ **Run job** runs another job as a task, which is how you compose a large workflow from smaller reusable ones.
- ◆ **It keeps ownership and scheduling separate**, so each sub-job can also be run on its own.
- ◆ **JAR and Spark Submit run on classic compute only**. Serverless is not supported for them.
- ◆ **Spark Submit is classic jobs only**, not even classic all-purpose.

ON THE EXAM

Run job is the answer when a scenario describes a workflow that must also be runnable independently by another team.

Tasks with no dependency between them **run in parallel**. That is a feature, and it is also how people accidentally create race conditions.

Shapes in the graph

- ◆ **Fan-out:** one task, several dependents. They all start when it finishes, in parallel.
- ◆ **Fan-in:** one task depending on several. It waits for all of them.
- ◆ **A diamond** is both: split, work in parallel, then converge.
- ◆ **No cycles.** It is a directed acyclic graph, so a task cannot depend on something downstream of itself.

ON THE EXAM

A fan-in task is the natural place for a validation step, because it is the first point where everything upstream is known to have succeeded.

Recommended compute by task type

TASK	RECOMMENDED	ALSO SUPPORTED
Notebook, Python script, Python wheel	Serverless jobs	Classic jobs, classic all-purpose
SQL, dbt	Serverless SQL warehouse	Pro SQL warehouse
Pipeline	Serverless pipeline	Classic pipeline
JAR	Classic jobs	Classic all-purpose
Spark Submit	Classic jobs	Classic jobs only

ON THE EXAM

Serverless is the recommendation nearly everywhere. The exceptions are JAR and Spark Submit, which need classic compute.

Sharing compute between tasks

- ◆ **Several tasks in one job can use the same compute resource.**
- ◆ **A shared job cluster starts once** and is reused, rather than each task paying start-up separately.
- ◆ **But it stays alive across the whole job**, including while a long unrelated task runs.
- ◆ **Serverless removes the trade-off**, since there is no start-up cost to amortise and no idle to pay for.

ON THE EXAM

With classic compute, sharing a cluster trades start-up cost for idle cost. Which wins depends on the shape of the DAG.

Parameters

- ◆ **Job parameters** are defined once on the job and available to every task in it.
- ◆ **Task parameters** are set on an individual task.
- ◆ **A notebook reads them as widgets**, using `dbutils.widgets.get`.
- ◆ **Dynamic value references** supply runtime values such as the job id, run id and start time.

ON THE EXAM

Parameters are what make one job definition serve dev, test and prod. Module 21 uses the same idea in bundles.

Passing values between tasks

- ◆ **Task values let a task return a result** into the job environment for later tasks to read.
- ◆ **Set one in a notebook** with `dbutils.jobs.taskValues.set`, and read it downstream with `get`.
- ◆ **They are what If/else conditions test**, alongside job parameters and dynamic values.
- ◆ **They are small values, not datasets.** Pass a row count or a status, not a DataFrame.

ON THE EXAM

Writing to a table is how you pass data between tasks. Task values are for passing a decision.

Notifications and monitoring

- ◆ **Notifications can fire on start, success, failure**, and on a duration threshold being exceeded.
- ◆ **They can be set per job or per task**, so a critical task can alert separately.
- ◆ **Destinations include email** and integrations such as Slack or PagerDuty.
- ◆ **The `system.lakeflow` schema** holds job and task run records, queryable across the account.

ON THE EXAM

Joining the jobs system tables to the billing tables is how you attribute cost to individual jobs. Module 23 uses this.

Who the job runs as

- ◆ **A job runs as a specific identity**, and that identity's permissions determine what it can read and write.
- ◆ **Use a service principal for production**, not a person.
- ◆ **A job owned by an individual breaks when they leave**, or when their permissions change.
- ◆ **Job permissions are separate**: who can view, manage run, or manage the job itself.

ON THE EXAM

'Run as' answers a whole family of permission questions. If a job fails on access, check the identity before checking the grant.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Two tasks ran at the same time unexpectedly	No dependency was declared between them
A continuous pipeline in a job task	Not supported. Pipeline tasks need a triggered pipeline.
A dashboard must refresh after the data loads	A dashboard task, after the transformation tasks
A step must also be runnable on its own	A Run job task
Spark Submit on serverless	Not supported. Classic jobs only.
A value must reach a later task	Task values, via <code>dbutils.jobs.taskValues</code>
The job broke when an engineer left	It was running as a person, not a service principal

ON THE EXAM

Seven lines. The second is the cleanest elimination in the module.

A team has a Lakeflow declarative pipeline running in continuous mode, ingesting from Kafka. They want to add a downstream aggregation notebook that runs after the pipeline has processed each batch.

An engineer proposes a job with two tasks: a pipeline task pointing at the continuous pipeline, then a notebook task depending on it.

What is the problem?

- ☐ A Nothing; this is the standard way to sequence a pipeline and a notebook
- ☐ B Continuous pipelines cannot be used as a job task; the pipeline must be triggered, or the notebook must run separately
- ☐ C A notebook task cannot depend on a pipeline task
- ☐ D The pipeline task would need to be the last task in the job

◆ PAUSE AND DECIDE

What is the problem?

- A

 Nothing is wrong
It is standard for a triggered pipeline. The scenario specifies continuous, which is the one case it does not work for.
- B

 Continuous pipelines cannot be a job task
Correct. A pipeline task triggers a triggered pipeline. A continuous pipeline is already running, so there is nothing to trigger and it is not supported.
- C

 A notebook cannot depend on a pipeline task
It can. Any task can depend on any other; that is what the DAG is for.
- D

 The pipeline task must be last
Invented ordering rule. Task order comes entirely from declared dependencies.

THE SIGNAL

A continuous pipeline never 'finishes a batch', so there is no completion event for a downstream task to wait on.

A job has five tasks. Task A loads bronze. Tasks B, C and D each build a different silver table from bronze, and each takes about 20 minutes. Task E builds a gold table from all three.

Only B declares a dependency on A. C and D declare none. E depends on B, C and D. The job intermittently produces a gold table missing rows.

What is wrong?

- ☐ A E should depend only on B, since B already depends on A
- ☐ B C and D have no dependency on A, so they may start before bronze is loaded
- ☐ C The three silver tasks should run sequentially, not in parallel
- ☐ D Task E needs a retry policy to handle the missing rows

◆ PAUSE AND DECIDE

What is wrong?

A E should depend only on B

Backwards. Removing dependencies from E would let it run before C and D finish, making the problem worse and more frequent.

C Run the silver tasks sequentially

Would accidentally fix it by forcing an order, at the cost of 40 extra minutes. It treats the symptom, not the missing dependency.

B C and D lack a dependency on A

Correct. Undeclared means parallel, so C and D start immediately alongside A and read bronze while it is still being written.

D Add a retry policy to E

E is not failing. It succeeds and produces incomplete output, which retries cannot detect or correct.

THE SIGNAL

'Intermittently' is the tell. A race condition depends on timing, so it appears and disappears with no change to the code.

A job runs eight short notebook tasks in sequence, each taking about 90 seconds. Each task is configured with its own classic job cluster. The job takes nearly 45 minutes end to end.

The team is surprised, since the actual processing is around 12 minutes in total.

What is happening, and what is the best fix?

- ☐ A Each task is starting its own cluster; use a shared job cluster, or move to serverless
- ☐ B The tasks should run in parallel rather than in sequence
- ☐ C The clusters are undersized; increase the node type for each task
- ☐ D Combine all eight notebooks into a single notebook task

◆ PAUSE AND DECIDE

What is happening, and what is the best fix?

A Eight cluster starts; share a cluster or go serverless
Correct. Eight classic clusters at roughly four minutes each is about 32 minutes of pure start-up, which matches the gap exactly.

C The clusters are undersized
Processing is 12 minutes across eight tasks. The missing 33 minutes is not compute time, so faster nodes change almost nothing.

B Run them in parallel
The scenario says in sequence, which usually means they depend on each other. Parallelising would also still start eight clusters.

D Combine into one notebook
It would work by accident, since one task means one cluster. It also throws away per-task retries, monitoring and reuse.

THE SIGNAL

45 minutes wall clock, 12 minutes of work. Whenever that gap is large and the tasks are short, look at start-up before compute.

A job's first task loads new records and needs to tell later tasks how many arrived. If zero rows arrived, an expensive downstream rebuild should be skipped entirely.

An engineer proposes writing the count to a small Delta table, which the downstream task then reads to decide whether to proceed.

What is the more idiomatic approach?

- ☐ A The Delta table approach is correct; it is the only way to share state between tasks
- ☐ B Set a task value with `dbutils.jobs.taskValues`, and branch on it with an `If/else` condition
- ☐ C Use a job parameter, set by the first task at runtime
- ☐ D Combine both tasks into one notebook so the count stays in memory

◆ PAUSE AND DECIDE

What is the more idiomatic approach?

- A

The Delta table is the only way
It works, and 'only way' is wrong. It also creates a table whose sole purpose is passing one number between two tasks.
- C

A job parameter set at runtime
Job parameters are supplied when the run starts, not written by a task mid-run. Wrong direction.
- B

A task value, branched on with If/else
Correct. Task values exist to return a result into the job environment, and If/else conditions can test them directly.
- D

Combine into one notebook
Removes the ability to skip the expensive work as a separate task, which was the entire requirement.

THE SIGNAL

Task values pass a decision; tables pass data. A single row count is a decision, so it belongs in a task value.

Declare dependencies, not order. Choose compute per task. And remember that **anything you did not connect runs in parallel.**